



Reinforcement Learning-Enhanced MPC for Sustainable Greenhouse Climate Management

Masoud Pourghavam

► To cite this version:

Masoud Pourghavam. Reinforcement Learning-Enhanced MPC for Sustainable Greenhouse Climate Management. The 12th International Conference on Electrical Engineering, Computer, Mechanics and Artificial Intelligence, 2025, Mashhad, Iran. <hal-05401361>

HAL Id: hal-05401361

<https://hal.science/hal-05401361v1>

Submitted on 5 Dec 2025

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



HAL Authorization

Reinforcement Learning-Enhanced MPC for Sustainable Greenhouse Climate Management

Masoud Pourghavam

Mechanical Engineering, University of Tehran
masoud.pourghavam@ut.ac.ir

Abstract

Greenhouse climate control presents a complex challenge involving nonlinear dynamics, environmental uncertainty, and the need for economically efficient decisions. This paper proposes a novel hybrid control framework—RL-Guided MPC—that integrates Reinforcement Learning (RL) with Model Predictive Control (MPC) to enhance economic performance in autonomous greenhouse management. The approach uses a trained Soft Actor-Critic RL agent to define both a terminal cost and terminal region constraint, thereby guiding the MPC's decision-making without requiring long prediction horizons. Simulations were conducted under both deterministic and stochastic conditions using a validated greenhouse model. Comparative evaluations with standalone MPC and RL controllers reveal that RL-Guided MPC consistently outperforms both, especially at shorter horizons, by effectively balancing long-term economic objectives with short-term system constraints. The framework retains MPC's strengths in constraint satisfaction while leveraging RL's ability to learn optimal control strategies from experience. While the RL agent alone underperformed in cumulative reward metrics due to constraint handling limitations, its integration into MPC notably improved overall system performance. These results demonstrate the potential of combining data-driven policies with model-based control for energy-efficient, economically optimal greenhouse operation.

Keywords: Greenhouse climate control, Reinforcement Learning, Model Predictive Control, Economic optimization

1 Introduction

Greenhouses play a crucial role in sustainable agriculture by providing a controlled environment that enhances crop yield while promoting efficient resource usage. To manage these systems effectively, especially under varying external weather conditions, advanced control methods are needed to dynamically regulate key climate variables such as temperature, humidity, and CO₂ concentration. However, traditional control techniques often fall short when it comes to balancing real-time operational decisions with long-term economic performance.

Reinforcement Learning (RL) and Model Predictive Control (MPC) have both emerged as promising tools to tackle these challenges. RL is well-suited for dealing with uncertainty and learning optimal strategies through experience, while MPC offers a robust, model-based approach that enforces constraints and adapts decisions over time. Previous studies have largely investigated RL and MPC independently. For instance, van Laatum et al. (2024) introduced an open-source environment to evaluate various RL methods for greenhouse control, and Morcego et al. (2023) developed a deep deterministic policy gradient-based RL controller and compared its performance with that of an MPC-based solution. On the MPC side, Boersma et al. (2022) proposed a robust MPC controller that improved upon traditional implementations. Yazdi et al. (2024) propose a hybrid control strategy that combines MPC with reinforcement learning to enhance quadruped locomotion in dynamic environments. Their results show improved velocity tracking and task efficiency using the RL-MPC approach compared to pure MPC. A more integrated approach was explored by Mallick et al. (2025), where MPC was used as a function approximator within an RL framework.

The core of the greenhouse control problem lies in economic optimization. While conventional MPC typically targets steady-state tracking of economically favorable conditions, such static approaches may not suit greenhouse operations, which benefit more from time-varying adjustments to climate conditions for better economic returns. Economic MPC, which directly incorporates economic costs into the control objective rather than focusing on stabilizing to a fixed reference, is a better fit. Still, many implementations of economic MPC rely on steady-state or reference trajectories to ensure stability and performance (as discussed in Amrit et al., 2011; Risbeck and Rawlings, 2020; Angeli et al., 2012).

This paper proposes a novel hybrid approach—**RL-Guided MPC**—that leverages a trained RL policy to enhance the MPC framework. Specifically, the RL policy is used to define both the terminal cost, via a cost function approximator, and the terminal region constraint, based on policy rollouts. This combination enables the controller to maintain high performance even with shorter prediction horizons, improving computational efficiency without compromising control quality.

To evaluate the effectiveness of the proposed method, simulations were carried out in two settings: a deterministic environment and one with parametric uncertainty. The RL-Guided MPC controller is compared against traditional MPC and standalone RL in both cases.

The remainder of the paper is organized as follows: Section 2 introduces the greenhouse model, simulation scenarios, and the shared optimization problem. Sections 3 and 4 detail the RL agent and MPC controller, respectively. Section 5 presents the proposed RL-Guided MPC framework. Section 6 provides numerical results, and conclusions are drawn in Section 7.

Table 1. Physical meaning of input u , state x , and disturbance d .

inputs	meaning	states	meaning	disturbances	meaning
u_1	CO ₂ injection (mg m ⁻² s ⁻¹)	x_1	Dry weight (kg m ⁻²)	d_1	Solar radiation (W m ⁻²)
u_2	Ventilation (mm s ⁻¹)	x_2	Indoor CO ₂ (ppm)	d_2	outdoor CO ₂ (kg m ⁻³)
u_3	Heating (W m ⁻²)	x_3	Indoor temperature (°C)	d_3	Outdoor temperature (°C)
		x_4	Indoor humidity (%)	d_4	Outdoor humidity (kg m ⁻³)

2 Background

2.1 Greenhouse Model

The greenhouse and crop model adopted in this study is based on the formulation presented by Van Henten (1994). To implement it in discrete time, the system is discretized using a fourth-order Runge-

Kutta method, with a sampling interval of $\Delta t = 1800$ seconds (equivalent to 30 minutes). This discretization yields the following state-space representation:

$$x(k+1) = f(x(k), u(k), d(k), p) \quad (1)$$

The system runs in discrete time, where the time index is defined as $k \in \mathbb{Z}^+$. At each step, the system's state is represented by $x(k) \in \mathbb{R}^4$, the control input by $u(k) \in \mathbb{R}^3$, and the external weather disturbance by $d(k) \in \mathbb{R}^4$. All the parameters required by the model are grouped into the vector $p \in \mathbb{R}^{22}$. The specific values of p and the nonlinear function $f(\cdot)$ are explained in Boersma et al. (2022).

This work assumes that the full state $x(k)$ is available at every time step, so no state estimation is performed. The physical meanings of the states, inputs, and disturbances are listed in Table 1. The dataset used for training and simulation comes from the Venlow Greenhouse in Bleiswijk, covering January 30 to March 11, 2014. The weather conditions during this period are treated as deterministic and known for every time step. Since the time step is 30 minutes, the dataset spans 40 days and contains 1920 steps.

Two scenarios are analyzed in simulation: one deterministic and one stochastic. In real greenhouse systems, uncertainty arises due to imperfect weather forecasts, measurement errors, or actuator imprecision. Following the approach in Boersma et al. (2022), this uncertainty is modeled as parametric variation. The uncertain parameter vector is denoted by $\hat{p} \in \mathbb{R}^{22}$ and is assumed to be sampled from a uniform distribution:

$$\hat{p} \sim U(p(1 - \delta), p(1 + \delta)) \quad (2)$$

In this context, the vector p denotes the nominal parameter values, while δ specifies the percentage range used to define the bounds of the uniform distribution. Throughout all stochastic simulations described in Section 6, the value of δ is fixed at 5%.

For every run of the stochastic scenario, a fresh realization of the uncertain parameter vector $\hat{p}$ is drawn from the distribution defined by Equation (2). The system is then simulated using the model described in Equation (1), substituting $p = \hat{p}$ during each rollout.

2.2 Optimization Problem

The objective of optimal greenhouse control is to sustain environmental conditions that promote healthy crop development—specifically regulating temperature, humidity, and CO₂ concentration—while keeping resource usage to a minimum. This regulation is performed by adjusting control inputs: heating (via pipe systems), ventilation (through window operation), and CO₂ enrichment (via injection). The overarching goal is to maximize crop yield while minimizing input costs and energy consumption.

Table 2. Pricing and penalty factors.

Symbol	Value	Unit
c_1	$1.906 \cdot 10^{-1}$	€ mg ⁻¹
c_3	$1.281 \cdot 10^{-1}$	€ J ⁻¹
r	20.93	€ kg ⁻¹
λ_2	$5 \cdot 10^{-5}$	€ (ppm m ²) ⁻¹
λ_3	$5 \cdot 10^{-3}$	€ (°C m ²) ⁻¹
λ_4	$5 \cdot 10^{-4}$	€ (% m ²) ⁻¹

Table 3. Input and output constraints and initial conditions.

Symbol	Value	Symbol	Value	Symbol	Value
$u_1^{\min}$	0	$u_1^{\max}$	1.2	$u_1(0)$	0
$u_2^{\min}$	0	$u_2^{\max}$	7.5	$u_2(0)$	0
$u_3^{\min}$	0	$u_3^{\max}$	150	$u_3(0)$	50
$x_1^{\min}$	0	$x_1^{\max}$	□	$x_1(0)$	0.0035
$x_2^{\min}$	500	$x_2^{\max}$	1600	$x_2(0)$	0.001
$x_3^{\min}$	10	$x_3^{\max}$	20	$x_3(0)$	15
$x_4^{\min}$	0	$x_4^{\max}$	80%	$x_4(0)$	0.008

While determining the optimal harvest time can significantly impact overall profitability, this study simplifies the problem by assuming a fixed cultivation period of 40 days. This modeling choice is consistent with previous works such as Boersma et al. (2022), Morcego et al. (2023), and Mallick et al. (2025). At the end of this period, the crop is harvested and sold, marking the end of the production cycle.

The Economic Profit Indicator (EPI) is then calculated by subtracting the total heating and CO₂ injection costs from the revenue generated through the crop sale. This is computed as:

$$EPI = r \cdot x_1(t_f) - \sum_{k=0}^{t_f} (c_1 u_1(k) + c_3 u_3(k)) \Delta t \quad (3)$$

In the above expression, t_f refers to the final time, which is fixed at 40 days. The parameter c_1 is the cost coefficient associated with CO₂ injection, while c_3 corresponds to the cost of heating. The term r denotes the revenue coefficient linked to the sale of lettuce, and Δt represents the sampling interval. The numerical values for these parameters are provided in Table 2.

The Economic Profit Indicator (EPI) serves as the objective function for the greenhouse control task. However, solving this objective directly using either Model Predictive Control (MPC) or Reinforcement Learning (RL) poses a challenge due to the sparse reward structure, where feedback (i.e., profit) is only received at the end of the 40-day period.

To address this issue, an economic stage cost is introduced and evaluated at each time step, defined as:

$$l_e(x(k), u(k)) = -r(x_1(k) - x_1(k-1)) + (c_1 u_1(k) + c_3 u_3(k)) \Delta t \quad (4)$$

When summed over the entire 40-day growing period, the stage cost defined in Equation (4) yields a value that is equivalent to the EPI in Equation (3), but with a negative sign. To ensure that the system operates under feasible and realistic conditions, state constraints are imposed. These include bounds on internal temperature, humidity, and CO₂ concentration, which must remain between specified minimum and maximum values.

However, because the environment is subject to uncertainties, these constraints cannot be strictly enforced in practice. Therefore, they are handled as soft constraints by adding penalty terms to the cost function whenever violations occur.

Taking these penalties into account, the stage cost is extended and redefined as:

$$l(x(k), u(k)) = l_e(x(k), u(k)) + g_2(x_2(k)) + g_3(x_3(k)) + g_4(x_4(k)) \quad (5)$$

In this formulation, the terms g_1 , g_2 , and g_3 represent the **penalty functions** applied to violations of the indoor CO₂ concentration, temperature, and humidity constraints, respectively. These functions are defined piecewise as follows:

$$g_i(x_i(k)) = \begin{cases} \lambda_i(x_i(k) - x_i^{\max}), & \text{if } x_i(k) > x_i^{\max} \\ \lambda_i(x_i^{\min} - x_i(k)), & \text{if } x_i(k) < x_i^{\min} \\ 0 & \text{otherwise} \end{cases} \quad (6)$$

The penalty weights λ_2 , λ_3 , and λ_4 used in the functions g_2 , g_3 , and g_4 are listed in Table 2. The corresponding minimum and maximum bounds for each state variable, denoted by $x_i^{\min}$ and $x_i^{\max}$, are specified in Table 3.

In addition to these state constraints, the control inputs—which include heating, ventilation, and CO₂ injection—are also subject to hard limits. Each input is constrained within its respective range, as defined in Table 3.

To avoid sudden or unrealistic changes in actuation, rate constraints are also enforced. For each control input $i \in \{1, 2, 3\}$, the maximum allowable change in input between time steps is limited by

$\delta u_i^{\max} = \frac{u_i^{\max}}{10}$. Considering the objective function in (5), along with the state and rate constraints, the optimal greenhouse control task is described by the following optimization problem:

$$\begin{aligned} \min_{u(\cdot)} \quad & \sum_{k=0}^{t_f} l(x(k), u(k)) \\ \text{s.t.} \quad & x(k+1) = f(x(k), u(k), d(k), p) \\ & u_{\min} \leq u_i(k) \leq u_{\max} \\ & |u(k) - u(k-1)| \leq \delta u^{\max} \end{aligned} \quad (7)$$

The Reinforcement Learning (RL) agent, the Model Predictive Controller (MPC), and the RL-Guided MPC all aim to solve the optimization problem defined in Equation (7). Their performance is ultimately assessed based on the cumulative cost specified in part (7a) of the formulation. For all simulation runs—across both the deterministic and stochastic environments—the initial conditions (as listed in Table 3) were held constant throughout each episode.

3 Reinforcement Learning

Actor-critic methods combine the strengths of both policy-based and value-based reinforcement learning techniques, offering a stable and sample-efficient approach to training. In this framework, the actor is responsible for learning the control policy, while the critic evaluates the actions taken by estimating their long-term value. This two-part structure enables the policy to improve through both direct interaction with the environment and evaluative feedback from the critic.

A major benefit of actor-critic algorithms is their natural suitability for continuous state and action spaces, unlike methods such as Q-learning, which are more effective in discrete settings (Sutton and Barto, 2020). This makes them particularly advantageous for controlling complex systems, such as greenhouse climate regulation. The specific actor-critic algorithm used in this study is the Soft Actor-Critic (SAC) algorithm, introduced by Haarnoja et al. (2018). SAC enhances standard actor-critic frameworks by incorporating an entropy term into the reward function, promoting a balance between exploration of new actions and exploitation of known strategies.

Table 4. Hyperparameters for the RL agent.

Parameter	Value
Training episodes	100
Warm-up episodes	9
Hidden layers	2
Neurons per hidden layer	128
Batch size	1024
Learning rate	$5 \cdot 10^{-3}$
Buffer size	100,000
Activation function	ReLU
Discount factor	0.95

As a result, the agent learns a policy denoted by $\pi_\theta(s(k))$, which maps the current observation $s(k)$ to an action, and is parameterized by θ . The inclusion of an entropy term in the objective encourages randomness in action selection, promoting better exploration. The policy is optimized by balancing both the expected cumulative reward and entropy, which fosters the discovery of more diverse strategies.

A well-designed observation space is critical for effective learning. If the observation is too limited, the agent may fail to develop useful control strategies. On the other hand, an overly complex observation space may introduce noise and make learning difficult. In practical greenhouse operations, certain variables—such as the current dry weight of the crop—are not directly measurable by growers. Nevertheless, this study assumes that the value of dry weight, represented as $x_1(k)$, is available to the RL agent, as well as to the MPC and RL-Guided MPC controllers.

Along with $x_1(k)$, the agent receives other measurable state information, including indoor temperature $x_2(k)$, CO₂ concentration $x_3(k)$, and humidity $x_4(k)$, which are commonly obtained from greenhouse sensors. The previous control input $u(k-1)$ is also included in the observation vector, which is essential due to the rate constraint specified in Equation (7d). This constraint limits the change in control inputs between time steps, so knowledge of $u(k-1)$ helps the agent make feasible decisions.

Additionally, the current weather disturbance $d(k)$ is part of the observation, just as it is for the MPC and RL-Guided MPC schemes. Finally, the agent is made time-aware by explicitly including the current time step k in its observation. This allows the policy to adapt its behavior based on the progression of the growth cycle. Without knowledge of k , the agent would be unable to differentiate between early and late phases of the growing season—a crucial distinction for optimal control.

Based on all of the above, the observation space used by the agent is defined as:

$$s(k) = (x(k), u(k-1), d(k), k) \quad (8)$$

To make sure that the control input $u(k)$ satisfies the constraints outlined in Equation (7), the agent's action—denoted by $a(k) = \pi_\theta(s(k))$ and bounded such that $a \in [-1, 1]^3$ —is treated as an incremental adjustment to the previous control input $u(k-1)$. This adjustment is applied as follows:

$$u(k) = \max(u_{\min}, \min(u_{\max}, u(k-1) + a(k) \cdot \delta u_{\max})) \quad (9)$$

The reward function used to train the RL agent is derived from the same cost function defined in Equation (5), but with its sign inverted. This setup ensures that the RL agent maximizes reward, while still aligning with the overall optimization objective. Using a consistent formulation allows for a fair comparison across the RL, MPC, and RL-Guided MPC approaches.

A key hyperparameter in this framework is the discount factor, denoted by γ . For long-horizon problems, setting $\gamma = 1$ allows the agent to take the entire growing period into account, optimizing for long-term economic performance. However, such a setting can cause instabilities during training. To mitigate this, the discount factor was set to $\gamma = 0.95$, which led to improved policy learning and more stable convergence. The complete list of hyperparameters used for the RL agent is provided in Table 4.

Two RL agents were trained for this study—one tailored to the deterministic environment and the other to the stochastic environment. Both agents used the same set of hyperparameters; the only distinction between them was the training environment and, consequently, the datasets they encountered during training.

4 Model Predictive Control

The Model Predictive Control (MPC) controller implemented in this work is formulated to solve the optimization problem described in Equation (7). At each time step k_0 , the controller computes the optimal control sequence by solving the following problem:

$$\begin{aligned}
 \min_{u(k)} \quad & \sum_{k=k_0}^{k_0+N_p} l(x(k), u(k)) \\
 \text{s.t.} \quad & x(k+1) = f(x(k), u(k), d(k), p) \\
 & u_{\min} \leq u(k) \leq u_{\max} \\
 & |u(k) - u(k-1)| \leq \delta u_{\max} \\
 & x(k_0) = x_{k_0}
 \end{aligned} \tag{10}$$

In the above formulation, N_p denotes the prediction horizon, and x_{k_0} represents the system's current state at time step k_0 . After solving the optimization problem, only the first control input in the computed sequence is applied. Then, at the next time step, the optimization problem is re-solved using the updated system state. This receding horizon approach is repeated at every time step until the end of the cultivation period.

To enhance the solver's efficiency and maintain feasibility, the optimization is warm-started using the previous solution. The entire optimization process is implemented using the open-source CasADi framework (Andersson et al., 2019) along with the IPOPT solver (Wächter and Biegler, 2006).

Two distinct MPC controllers are implemented—one designed for the deterministic environment and the other for the stochastic environment. Both controllers are based on the same formulation described in Equation (10). The only difference lies in the treatment of the indoor humidity constraint $x_4^{\max}$: for the stochastic case, this constraint is tightened from 80% to 78%. This tightening helps to reduce the risk of constraint violation in the presence of parametric uncertainty during simulation.

5 RL-Guided MPC

Below, we present the RL-Guided MPC framework, which integrates the agent developed in Section 3 into the MPC optimization discussed in Section 4. We begin by explaining the derivation of the terminal cost from the critic. Then, we detail the use of the actor to establish the terminal region constraint. Lastly, we demonstrate how the actor assists in generating a warm-start initial guess for the RL-Guided MPC optimization problem.

5.1 Terminal Cost

In the RL-Guided MPC framework, the terminal cost is derived directly from the actor network of the SAC agent introduced in Section 3. By utilizing the learned policy π_θ , several closed-loop trajectories are simulated. These trajectories are then used to train a cost function approximator J_ϕ , which estimates the expected return for any given state. This method allows the RL-Guided MPC to consider long-term outcomes throughout the entire growth period without needing to extend the prediction horizon. The approximator is trained using expected return learning based on a nominal state trajectory $x^n(k)$, 1000 time steps are uniformly sampled from the interval $\{0, 1, \dots, t_f\}$. For each selected time step $k^{(i)}$, an initial state $x(k^{(i)})$ is drawn from a uniform distribution within a range specified by the nominal trajectory.

$$x(k^{(i)}) \sim U(x_n^{\min}(k^{(i)}), x_n^{\max}(k^{(i)})) \tag{11}$$

$$\begin{aligned}
 x_{\min}^n(k) &= x^n(k)(1 - \sigma) \\
 x_{\max}^n(k) &= x^n(k)(1 + \sigma)
 \end{aligned} \tag{12}$$

The parameter σ , set to 50%, controls the dispersion of the initial sampled states around the reference trajectory. For each sampled state $x(k^{(i)})$, the corresponding agent observation $s(k^{(i)})$, as defined in Equation (8), is generated. Using the policy π_θ , a closed-loop trajectory is then produced, continuing through to the end of the current growth cycle t_f . The cumulative cost for each of these trajectories is subsequently calculated as follows:

$$J_{\pi_{\theta}}(s(k^{(i)})) = \sum_{k=k_s}^{t_f} l(x(k), \pi_{\theta}(s(k))) \quad (13)$$

The cumulative cost associated with each trajectory is stored along with the corresponding dry weight $x_1(k^{(i)})$ and sampling time $k^{(i)}$, forming the dataset D . This dataset is then split into training and validation subsets, with 80% allocated for training and the remaining 20% for validation. A neural network parameterized by ϕ is trained to approximate the cost function $J_{\phi}^{\%}$, using $x_1(k^{(i)})$ and $k^{(i)}$ as inputs, and the cost values $J_{\pi_{\theta}}(s(k^{(i)}))$ as targets. Training is carried out by minimizing the following loss function using the Adam optimizer (Kingma and Ba, 2017):

$$L(\phi, D) = \frac{1}{N_s} \sum_{i=1}^{N_s} \left(J_{\phi}^{\%}(x_1(k^{(i)}), k^{(i)}) - J_{\pi_{\theta}}(s(k^{(i)})) \right)^2$$

The hyperparameter settings for the cost function approximator are listed in Table 5. This approximator is subsequently incorporated as an additional term in the objective function of the MPC optimization problem described in Equation (10). As a result, at each time step k_0 , the cost function for the RL-Guided MPC is defined as follows:

$$\sum_{k=k_0}^{k_0+N_p} l(x(k), u(k)) + J_{\phi}^{\%}(x_1(k_0+N_p), k_0+N_p) \quad (14)$$

5.2 Terminal region constraint and initial guess

At the initial time step, the terminal region constraint and the initial guess are generated based on the initial conditions provided in Table 3 and the policy of the actor. The agent's initial observation $s(0)$ is formed using these initial conditions. The policy π_{θ} is then applied to compute the closed-loop trajectory from the starting time step up to $N_p + 1$:

$$\{s(0), s(1), K, s(N_p + 1)\} \quad (15)$$

along with the corresponding control input trajectory:

$$\{u(0), u(1), K, u(N_p)\} \quad (16)$$

Where $u(k) = \pi_{\theta}(s(k))$. Using the closed-loop trajectory computed in the observation space as defined in (15), the corresponding state trajectory is then derived:

$$\{x(0), x(1), K, x(N_p + 1)\} \quad (17)$$

The center of the terminal region constraint for the subsequent optimization problem is set to the final state of this trajectory, specifically $x_f = x(N_p + 1)$. Accordingly, the terminal region constraint for this initial optimization problem is defined as:

$$[(1 - \varepsilon)x_f, (1 + \varepsilon)x_f] \quad (18)$$

The parameter ε , represented as a percentage, specifies the dimensional bounds of the terminal region centered at x_f . To facilitate a warm-start for the ensuing optimization problem, the terminal N_p elements of the control input trajectory from equation (16) are employed as the initial estimate. This approach for defining the terminal region constraint and initializing the guess is exclusively applied at the initial time step. For all subsequent time steps, the construction of both the terminal region constraint and the initial guess relies on the optimized solution obtained from the preceding time step, combined with guidance from the actor's policy. The resulting solution of the RL-Guided MPC optimization problem at each discrete time k_0 is denoted as:

$$\{u(k_0), u(k_0 + 1), K, u(k_0 + N_p - 1)\} \quad (19)$$

with the associated predicted state trajectories represented as

$$\{x(k_0 + 1), x(k_0 + 2), K, x(k_0 + N_p)\} \quad (20)$$

The terminal region constraint and the initial guess for the subsequent optimization problem are formulated based on the current solutions in conjunction with the actor's policy. Beginning from the terminal predicted state $x(k_0 + N_p)$, the corresponding agent observation $s(k_0 + N_p)$ is generated, after which the policy π_θ is employed to determine the center of the terminal region for the upcoming optimization problem:

$$x_f = f(x(k_0 + N_p), \pi_\theta(s(k_0 + N_p)), d(k_0 + N_p), p) \quad (21)$$

The terminal region constraint for the subsequent optimization problem is established as specified in Equation (18). The initial guess for the forthcoming optimization problem is subsequently formulated as:

$$\{u(k_0 + 1), \dots, u(k_0 + N_p - 1), \pi_\theta(s(k_0 + N_p))\} \quad (22)$$

Two distinct RL-Guided MPC controllers were designed: one tailored for deterministic settings and the other for stochastic environments. The primary distinction between them lies in the policy employed to generate the terminal region constraint and the initial guess. Additionally, in the stochastic scenario, the indoor humidity constraint $x_4^{\max}$ is conservatively tightened to 78% to accommodate model uncertainties, thereby mitigating the likelihood of constraint violations, consistent with the approach detailed for the stochastic MPC in Section 4.

6 Results

The effectiveness of the RL agent, traditional MPC, and RL-Guided MPC was assessed across both deterministic and stochastic settings. Additionally, results for an alternative variant of the RL-Guided MPC are included for the deterministic environment. Within each scenario, the three approaches are evaluated and compared using the Energy Performance Indicator (EPI), as defined in Equation (3), alongside the cumulative reward metric, represented by the negation of Equation (7a). For clarity and conciseness in the figures presented in this section, the RL-Guided MPC is referred to simply as 'RL-MPC'.

6.1 Deterministic case

In the deterministic scenario, an additional variant of the RL-Guided MPC, referred to as 'RL-MPC-VFO', is evaluated. This variant mirrors the standard RL-MPC except for the exclusion of the terminal region constraint from the optimization formulation. The simulation outcomes for the deterministic setting are illustrated in Figure 1. Regarding the EPI metric, both MPC and RL-Guided MPC demonstrate comparable performance across varying prediction horizons, with RL-Guided MPC showing a modest advantage at shorter horizons. Conversely, the standalone RL agent underperforms significantly relative to both approaches. Notably, the RL-Guided MPC variant that omits the terminal region constraint exhibits performance closely aligned with that of the conventional MPC.

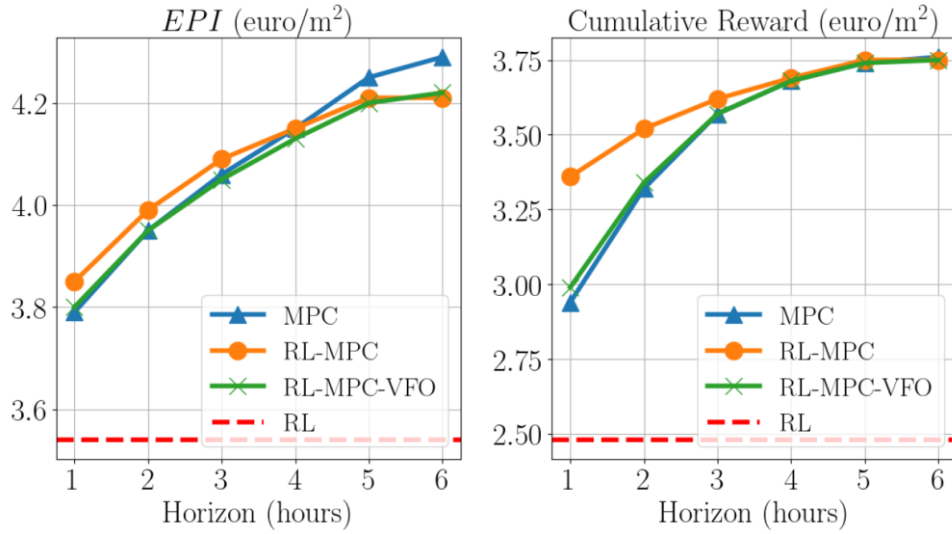


Figure 1. Comparison of EPI and cumulative reward between RL, MPC, and RL-Guided MPC, at different prediction horizons, for the deterministic case.

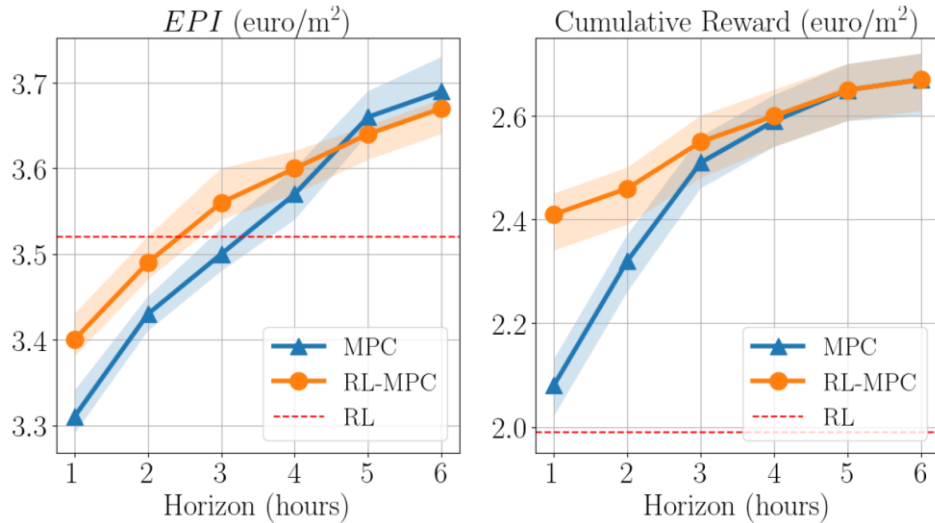


Figure 1. Comparison of EPI and cumulative reward between RL, MPC, and RL-Guided MPC, at different prediction horizons, for the stochastic case.

While evaluating controllers based on the Energy Performance Indicator (EPI) provides valuable insights, the primary objective of all controllers is to maximize the cumulative reward. In this regard, a pronounced divergence becomes evident between MPC and RL-Guided MPC at shorter prediction horizons, with RL-Guided MPC demonstrating a marked superiority. This advantage arises from its capacity to incorporate the learned policy, effectively capturing long-term system dynamics. However, as the prediction horizon extends beyond four hours, the cumulative reward differences between the two methods converge, yielding comparable performance. Meanwhile, the RL agent consistently underperforms, largely due to its limited capability in managing system constraints. In contrast, both MPC and RL-Guided MPC are explicitly formulated to handle constraints, enabling them to operate efficiently near constraint boundaries without incurring penalties. The RL-Guided MPC variant that omits the terminal region constraint exhibits performance akin to standard MPC, underscoring the critical role of this constraint within the RL-Guided MPC framework. It facilitates more effective guidance of the optimization process, reducing reliance solely on the learned cost function approximator.

6.2 Stochastic case

In the stochastic scenario, 30 distinct realizations of the parameter $\hat{p}$ as defined in equation (2) were generated. For each realization, the performance of the RL agent, MPC, and RL-Guided MPC was assessed, and the results were subsequently averaged. These aggregated outcomes are illustrated in Figure 2, where the shaded areas represent the span between the minimum and maximum observed values. The stochastic results largely mirror the trends observed in the deterministic setting, albeit with minor variations. In terms of the EPI metric, RL-Guided MPC consistently outperforms MPC at shorter prediction horizons. Interestingly, contrasting with the deterministic environment, the RL agent surpasses both MPC-based methods in EPI performance at these shorter horizons. Furthermore, aligning with the deterministic case, the cumulative reward evaluation reaffirms the superior performance of RL-Guided MPC over MPC at brief horizons, with a particularly pronounced advantage evident at the one-hour mark.

7 Conclusions

This study presents the RL-Guided MPC framework as a novel solution for autonomous greenhouse control, effectively merging the complementary advantages of Reinforcement Learning (RL) and Model Predictive Control (MPC). The approach involves training an RL agent and embedding its learned policy within the MPC optimization through the design of a terminal cost and terminal region constraint. This hybrid strategy harnesses the RL agent's ability to handle uncertainty alongside MPC's robust constraint management.

Simulation results, conducted under both deterministic and stochastic conditions, reveal that the RL-Guided MPC consistently surpasses the performance of standalone RL and conventional MPC, particularly at shorter prediction horizons. These findings highlight the benefit of integrating learned policies into MPC, enabling improved control without necessitating longer prediction horizons.

Nevertheless, it is noteworthy that the RL policy developed in this study underperformed relative to expectations, yielding lower cumulative rewards compared to the other methods across all prediction horizons. Future research will focus on exploring the impact of enhanced RL policies, with the anticipation that superior policy performance could drive significant gains in control effectiveness.

Moreover, leveraging an RL-derived policy is not the sole pathway to augment MPC. Alternative policy generation methods, such as expert knowledge-based rule controllers, could provide valuable complementary insights and potentially elevate system performance in greenhouse control applications.

References

- Amrit, R., Rawlings, J.B., and Angeli, D. (2011). Economic optimization using model predictive control with a terminal cost. *Annual Reviews in Control*, 35(2), 178–186. doi:10.1016/j.arcontrol.2011.10.011.
- Andersson, J.A.E., Gillis, J., Horn, G., Rawlings, J.B., and Diehl, M. (2019). CasADi – A software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11(1), 1–36. doi:10.1007/s12532-018-0139-4.
- Angeli, D., Amrit, R., and Rawlings, J.B. (2012). On average performance and stability of economic model predictive control. *IEEE Transactions on Automatic Control*, 57(7), 1615–1626. doi:10.1109/TAC.2011.2179349.
- Boersma, S., Sun, C., and Van Mourik, S. (2022). Robust sample-based model predictive control of a greenhouse system with parametric uncertainty. *IFAC-PapersOnLine*, 55(32), 177–182. doi:10.1016/j.ifacol.2022.11.135.
- Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Int. conference on machine learning (PMLR)*, 80:1861–1870.
- Kingma, D.P. and Ba, J. (2017). Adam: A method for stochastic optimization. URL <https://arxiv.org/abs/1412.6980>.
- Mallick, S., Airalidi, F., Dabiri, A., Sun, C., and De Schutter, B. (2025). Reinforcement learning-based model predictive control for greenhouse climate control. *Smart Agricultural Technology*, 10, 100751. doi:10.1016/j.atech.2024.100751.

- Morcego, B., Yin, W., Boersma, S., Van Henten, E., Puig, V., and Sun, C. (2023). Reinforcement Learning versus Model Predictive Control on greenhouse climate control. *Computers and Electronics in Agriculture*, 215, 108372. doi:10.1016/j.compag.2023.108372.
- Risbeck, M.J. and Rawlings, J.B. (2020). Economic Model Predictive Control for Time-Varying Cost and Peak Demand Charge Optimization. *IEEE Transactions on Automatic Control*, 65(7), 2957–2968. doi:10.1109/TAC.2019.2939633.
- Sutton, R.S. and Barto, A. (2020). Reinforcement learning: an introduction. Cambridge: MIT Press.
- Van Henten, E.J. (1994). Greenhouse climate management: an optimal control approach. Ph.D. thesis, Wageningen University.
- van Laatum, B., van Henten, E.J., and Boersma, S. (2024). Greenlight-gym: A reinforcement learning benchmark environment for greenhouse crop production control. URL <https://arxiv.org/abs/2410.05336>.
- Wächter, A. and Biegler, L.T. (2006). On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1), 25–57. doi:10.1007/s10107-004-0559-y.
- Yazdi, M.R.H., Pourghavam, M., and Bidari, R. (2024, December). Hybrid Control of Advanced Quadruped Locomotion: Integrating Model Predictive Control with Deep Reinforcement Learning. In 2024 12th RSI International Conference on Robotics and Mechatronics (ICRoM), 201–208. IEEE.